

ARK-457 — Cross-Context Authorization Replay (Confused Deputy)

RESULTS (recorded as executed)

Experiment ID: ARK-457

Series: ExecutionProof authorization-boundary corpus (enterprise-failure-mode phase)

Substrate: Classical software (no quantum hardware, no cryptography)

Execution window: 2026-07-18T00:19:46Z to 2026-07-18T00:19:46Z (UTC)

VERDICT: PASS

Summary

ARK-457 tested whether an authorization guard binds an authorization to its issuing context and refuses to let it be replayed under a different context — the classic **confused-deputy** failure mode.

Question: When an authorization is issued bound to a specific context (tenant, session, resource, audience, environment), can it be replayed to authorize execution under a **different** context? A correct guard must ALLOW only when the execution context matches the authorization's bound context **exactly** on every dimension, and DENY otherwise — including superficial “confusable” near-matches.

Answer: No — replay is refused. Both independently implemented guards (V1 JavaScript, V2 Python) allowed execution only when all five binding dimensions matched exactly (100%), and refused every cross-context replay (100%) — including confusable near-matches designed to fool a lenient comparator (trailing space, case flip, zero-width character, homoglyph). Mutual agreement was 100% across all 800 evaluation decisions.

Design

Structure: 8 arms × 100 scenarios per arm = **800 evaluation decisions**

Context tuple (binding dimensions): Every authorization is bound to a 5-dimension context tuple; execution presents its own tuple.

Dimension	Role	Example values
tenant	Isolation boundary / customer	tenant-0007
session	Session / token scope	sess-0002af
resource	Governed object	wallet-0031
audience	Intended API/service scope	api.payments , api.reporting
environment	Deployment stage	production , staging , sand- box , dr-failover

Decision procedure (both guards):

1. For each of the five dimensions: if the authorization's bound value and the execution's context value are not both strings, or are not **byte/code-point exactly equal** → **DENY** (authorization does not transfer across contexts).
2. If all five dimensions match exactly → **ALLOW**.

No normalization, no case folding, no whitespace trimming, no zero-width stripping, no homoglyph mapping. A confusable near-match is a **MISMATCH** → **DENY** by construction.

Arms:

```
| Arm | Condition | Mismatch | Expected |
|----|-----|-----|-----|
| 1 | BASELINE-exact-context-match | none (all 5 dims identical) | ALLOW |
| 2 | CROSS-TENANT | tenant differs | DENY |
| 3 | CROSS-SESSION | session differs | DENY |
| 4 | CROSS-RESOURCE | resource differs | DENY |
| 5 | CROSS-AUDIENCE | audience differs | DENY |
| 6 | CROSS-ENVIRONMENT | environment differs | DENY |
| 7 | MULTI-DIMENSION-mismatch | ≥ 2 dims differ | DENY |
| 8 | CONFUSABLE-near-match | exactly 1 dim differs by a superficial variant (trailing space, case flip, zero-width char, homoglyph) | DENY |
```

Primary Results

Pass Criteria (preregistered)

Let:

- **g_allow_exact** = ALLOW rate in Arm 1 (exact context match)
- **g_deny_crosscontext_min** = minimum DENY rate across Arms 2-8 (cross-context replay)
- **g_margin** = min(g_allow_exact, g_deny_crosscontext_min) – 0.05

Criteria:

- **C1:** g_allow_exact ≥ 0.95
- **C2:** g_deny_crosscontext_min ≥ 0.95
- **C3:** g_margin ≥ 0.90

Measured Values

Metric	Value	Criterion	Result
g_allow_exact	1.0000	≥ 0.95	✓ PASS
g_deny_crosscontext_min	1.0000	≥ 0.95	✓ PASS
g_margin	0.9500	≥ 0.90	✓ PASS

All three criteria met. VERDICT: PASS

Per-Arm Results

Arm	Condition	Expected	ALLOW	HOLD	DENY	V1-V2 Concordance
1	BASELINE-exact-context-match	ALLOW	100%	0%	0%	100% (100/100)
2	CROSS-TENANT	DENY	0%	0%	100%	100% (100/100)
3	CROSS-SESSION	DENY	0%	0%	100%	100% (100/100)
4	CROSS-RE-SOURCE	DENY	0%	0%	100%	100% (100/100)
5	CROSS-AUDIENCE	DENY	0%	0%	100%	100% (100/100)
6	CROSS-ENVIRONMENT	DENY	0%	0%	100%	100% (100/100)
7	MULTI-DIMENSION-mismatch	DENY	0%	0%	100%	100% (100/100)
8	CONFUS-ABLE-near-match	DENY	0%	0%	100%	100% (100/100)

Observations:

- Arm 1 (exact match): ALLOW 100% — legitimate same-context executions proceed
 - Arms 2-8 (cross-context): DENY 100% — authorization never transfers to a different context
 - Arm 8 (confusable): DENY 100% — the guard is not fooled by strings that would collapse under naive normalization
 - Perfect separation: no cross-context ALLOW, no false DENY of an exact match
 - Zero HOLD emissions — the guard treats context mismatch as a hard DENY, not a borderline case
-

Secondary Observations

Dual-Guard Concordance

- **Overall V1-V2 concordance:** 100.00% (800/800 agreements)
- Both independently written guards (JavaScript strict `!==`, Python strict `!=`) agreed on every decision across all 800 scenarios
- Confirms the exact-match context-binding procedure is clear, deterministic, and implementable without ambiguity
- All scenario values were held within the Unicode BMP so JavaScript (UTF-16) and Python (code-point) exact-equality comparisons are identical — concordance reflects decision logic, not encoding artifacts

No Cross-Context Leakage

- **Zero ALLOWs across 700 cross-context scenarios (arms 2-8):** not a single replayed authorization was accepted
 - The confusable arm (arm 8) is the sharpest test: every differing value was byte-different but engineered to merge under a lenient comparator; the strict guard refused all 100
-

Gates and Safeguards

Kill-Gate Calibration

- **Status:** PASS
- 88 calibration scenarios generated (mixed across all 8 arms)
- Context-replay effectiveness gate: all 88 scenarios verified by an independent structural oracle to genuinely encode their intended context relationship under exact equality
- V1-V2 concordance: 100% (88/88); V2: 12 ALLOW, 0 HOLD, 76 DENY (V1 identical)
- Gate threshold: $\geq 99\%$ concordance required to proceed → **exceeded**

Context-Replay Effectiveness Gate (per arm)

- **Status:** PASS on all 8 arms
- Every scenario in every arm verified by the structural oracle to genuinely encode its intended relationship (arm 1: 0 dims differ; arms 2-6: exactly one specified dim differs; arm 7: ≥ 2 dims differ; arm 8: exactly one dim differs by a confirmed confusable byte-difference that a naive normalizer would merge)
- Prevents ARK-455-style no-op defects where a scenario appears to encode a condition but is mathematically inert
- **800/800 scenarios validated** as effective before evaluation

Interpretation

What This Result Means

ARK-457 demonstrates that an authorization guard can bind an authorization to its issuing context and refuse to let it transfer to any other context:

1. **Exact context → allow.** When all five binding dimensions match, legitimate same-context execution proceeds (100% ALLOW).
2. **Any context difference → deny.** When any binding dimension differs, the guard refuses (100% DENY), regardless of how superficially small the difference appears.
3. **Not fooled by confusables.** Trailing spaces, case flips, zero-width characters, and homoglyphs are treated as genuine differences (100% DENY in arm 8).
4. **Unambiguous behavior.** Dual independent implementations agree on every decision (100% concordance).

This validates a core safety property against the confused-deputy problem: **an authorization is only valid in the exact context it was issued for.**

Real-World Relevance

Cross-context replay underlies many high-impact incidents:

- Multi-tenant SaaS isolation breaks (authorization for tenant A accepted for tenant B)
- OAuth/token audience confusion (token for one service accepted by another)
- Session token export/replay across sessions or devices
- Staging/sandbox credentials accepted in production
- Homoglyph/whitespace tricks defeating naive identifier comparison

ARK-457 shows that a guard using strict, non-normalizing context binding refuses every one of these replay classes.

Limitations and Scope

What this experiment tested:

- Whether a guard binds an authorization to its issuing context and refuses transfer to any other context
- Whether the comparison is strict enough to reject confusable near-matches
- Whether the decision procedure is clear enough for independent implementation

What this does NOT test:

- Real token formats, JWT/PASETO audience claims, or OAuth scope semantics
- Cryptographic binding (signatures, key confirmation, DPoP) — this is logical context binding only
- Network-level replay (TLS, nonce caches) or timing/expiry (covered by ARK-442/ARK-451)
- Unicode normalization policy design — here strict non-normalization is the tested behavior by construction
- Performance, latency, or throughput under load

Constraints: This is a classical/software boundary-logic testbed validating context-binding in isolation, not the end-to-end enterprise identity stack. Findings are scoped to the tested procedure, scenario generator, and arms.

Provenance and Integrity

LOCK Procedure

1. **Preregistration, dual guards, generator, runners, MANIFEST (SHA-256 hashes)**
committed **BEFORE** execution at commit `452f6ea` (lock timestamp 2026-07-18T00:17:33Z UTC), tagged `ark-457-v1.0-lock`
2. No scenario was generated or evaluated until after the LOCK commit
3. Results recorded AFTER execution complete

MANIFEST SHA-256 Hashes (locked files)

PREREGISTRATION.md:	9348475503b2d7b4b8d9e43140a936abecf7d-b96a9f59353ab013ced021cd88d
schemas/context_replay_scenario_schema.json:	6312a66df6ef5422ff19d7380aab001b-b76251b4dee986b3a84a2b44fe007e41
verifiers/v1_guard.js:	e2195a0cd76a9cd9a5b0e1e-be939aa68491752b0938db4d96b1678a2c5a75982
verifiers/v2_guard.py:	3620e43258450d8db-b571acec3c5b2a8a14ced85bf2b5e335aacfd51fde1058
generator/scenario_generator.py:	1f24a47eed82bfb8ff43a0bf0e12930d-c243e5ffc37e09da8f1db97e4947dbed
run_killgate.py:	f685f60ed57b737cbabcafa6c4bc2d542e58bf2f-d18760523462d9ae5c9e9c03
run_arms.py:	66de18cfd8b518ae073d-bed247ec1d19044d86be0e4acd925c4f5d5b7d532d58

Commits are not cryptographically signed. Provenance = commit history + MANIFEST SHA-256 hashes. This is an experimental testbed, not a production security claim.

Conclusion

ARK-457 answers the question **“Can an authorization be replayed across contexts?”** with a clear **NO**:

- Exact context match → ALLOW (100%)
- Any cross-context difference, including confusable near-matches → DENY (100%)
- Zero cross-context leakage
- Dual independent implementations agree (100%, 800/800)

The context-binding principle is validated: **an authorization is only valid in the exact context it was issued for.**

Verdict stands as executed: PASS.

Investigator: Derek Hone, Remnant Fieldworks Inc.

Executor: Abacus.AI autonomous agent (supervised)

Series: ExecutionProof™ authorization-boundary corpus

Trademarks: ExecutionProof™, ProofRecord™, VaultProof™, Verification Before Execution™, Proof Before Power™

If it cannot be verified, it cannot execute.